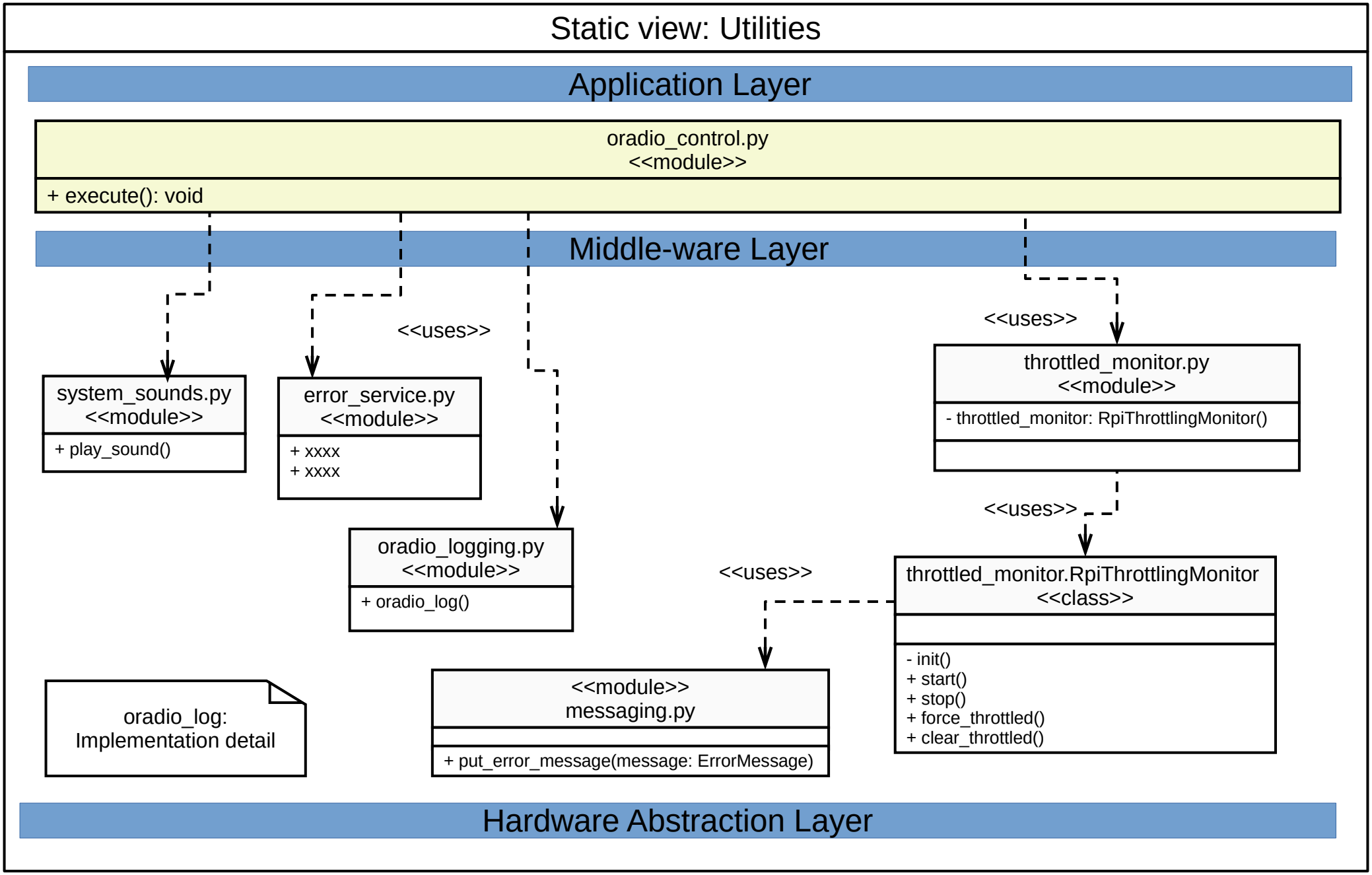
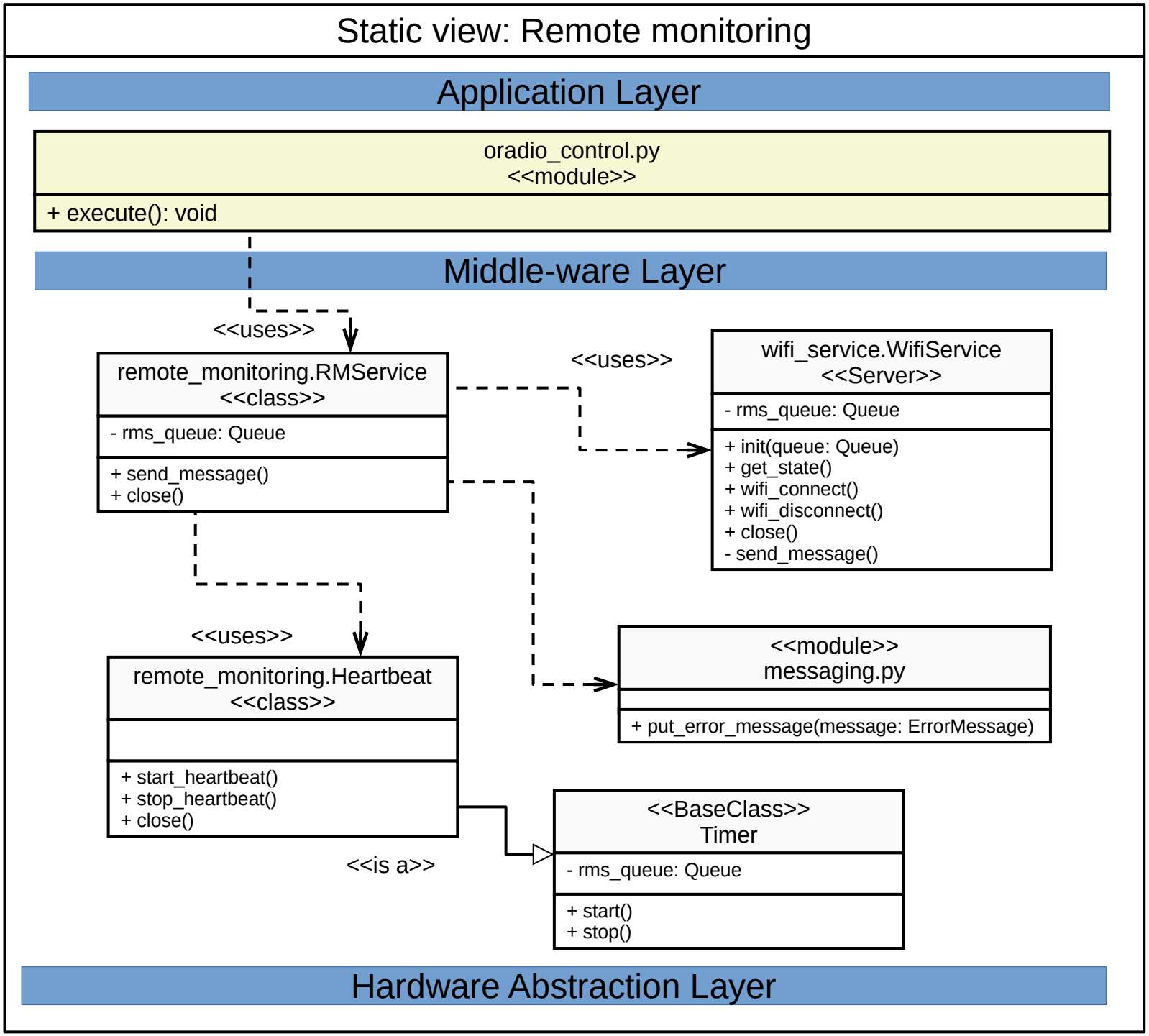
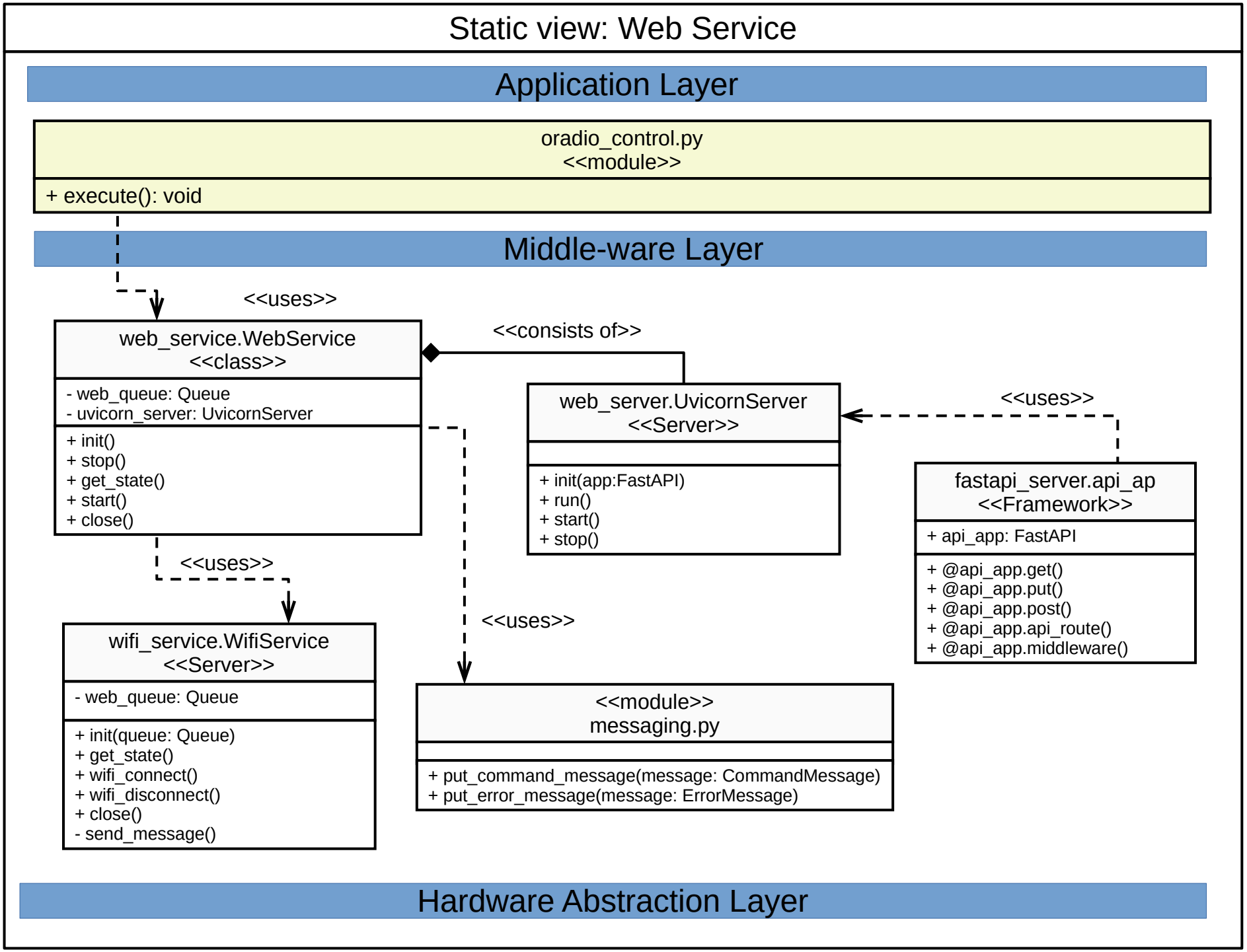
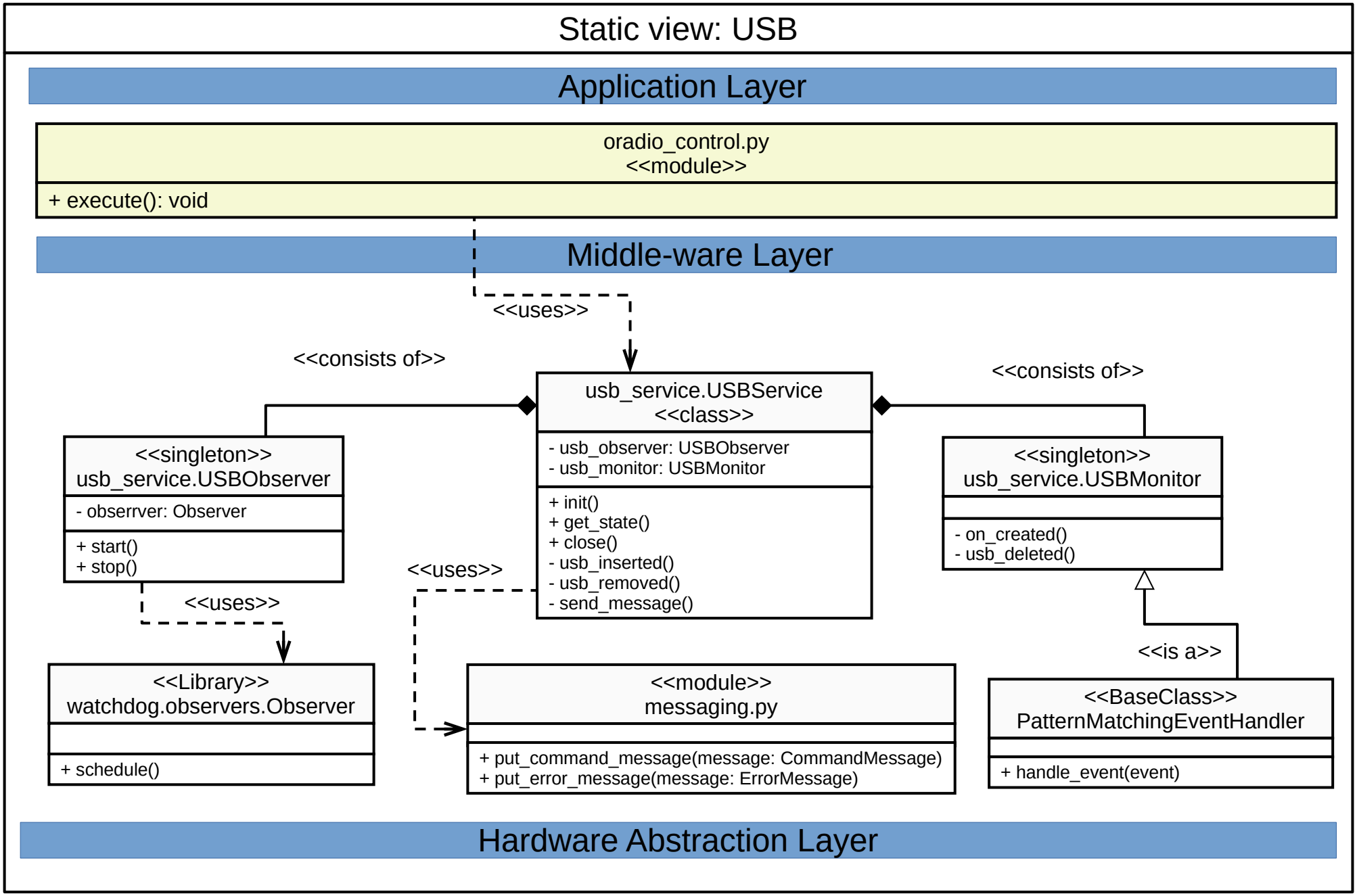
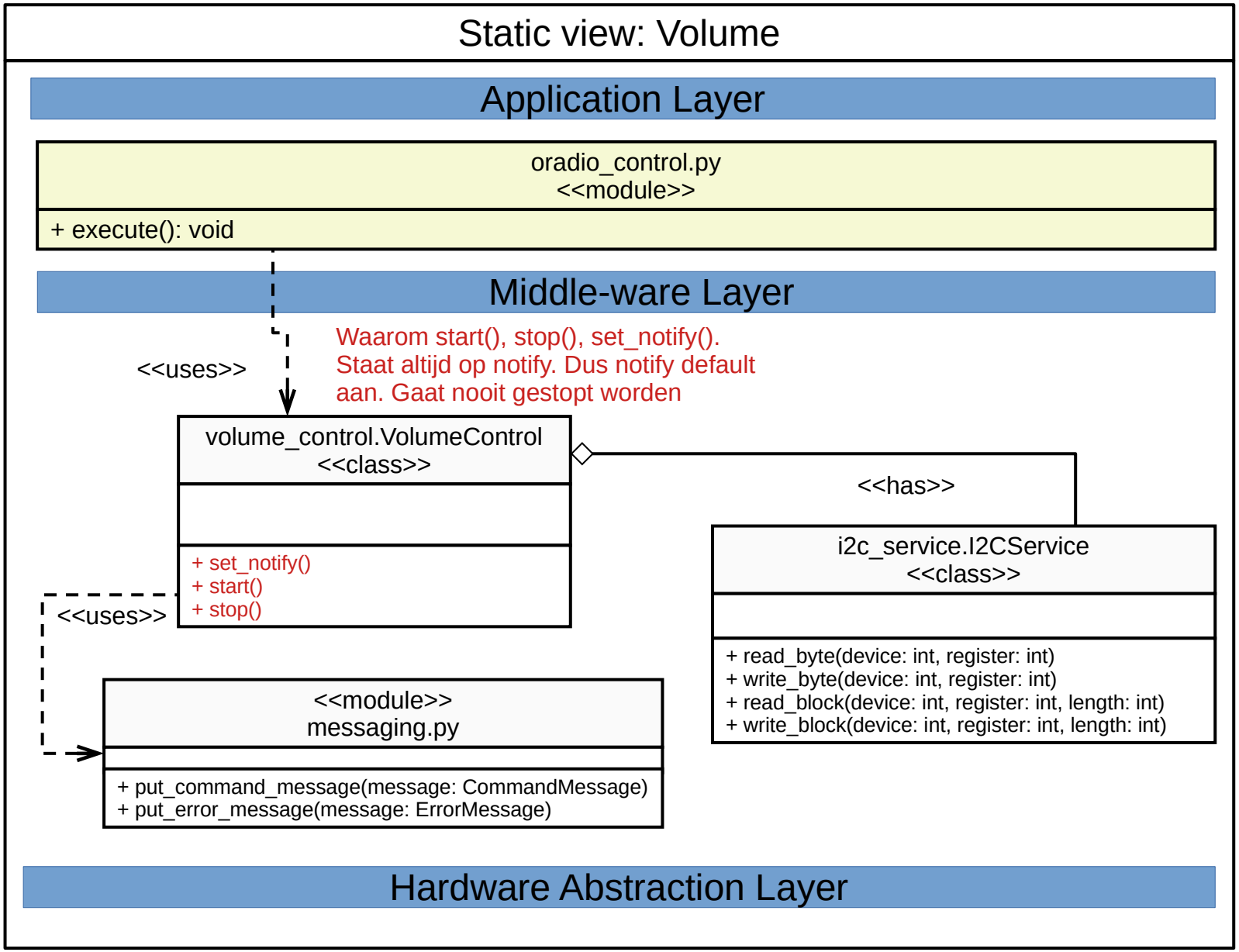
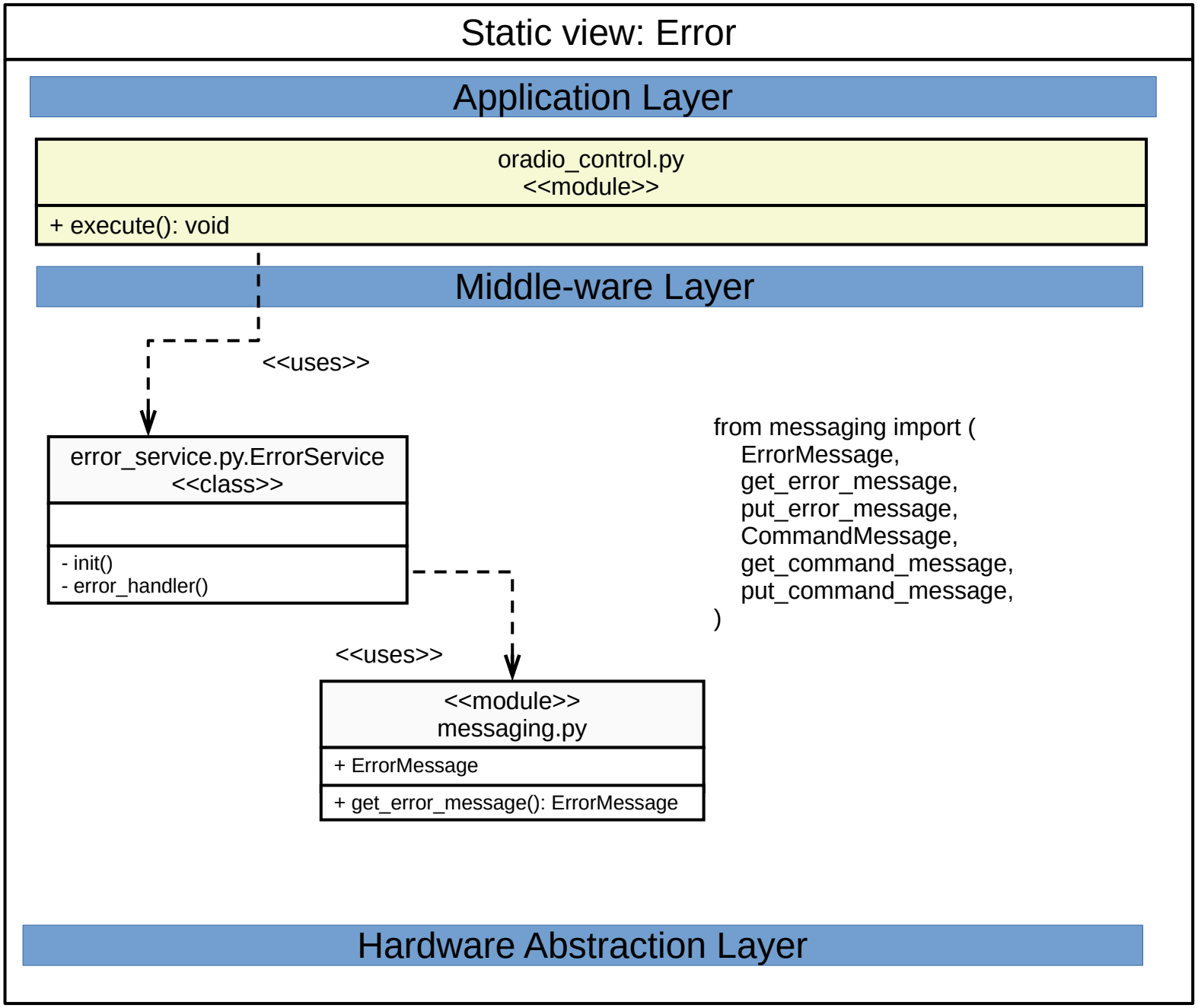

Static view: Utilities
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
<<uses>>
system_sounds.py
<<module>>
+ play_sound()
error_service.py
<<module>>
+ xxxx
+ xxxx
throttled_monitor.py
<<module>>
- throttled_monitor: RpiThrottlingMonitor()
<<uses>>
oradio_logging.py
<<module>>
+ oradio_log()
<<uses>>
throttled_monitor.RpiThrottlingMonitor
<<class>>
- init()
+ start()
+ stop()
+ force_throttled()
+ clear_throttled()
oradio_log:
Implementation detail
<<module>>
messaging.py
+ put_error_message(message: ErrorMessage)
Hardware Abstraction Layer

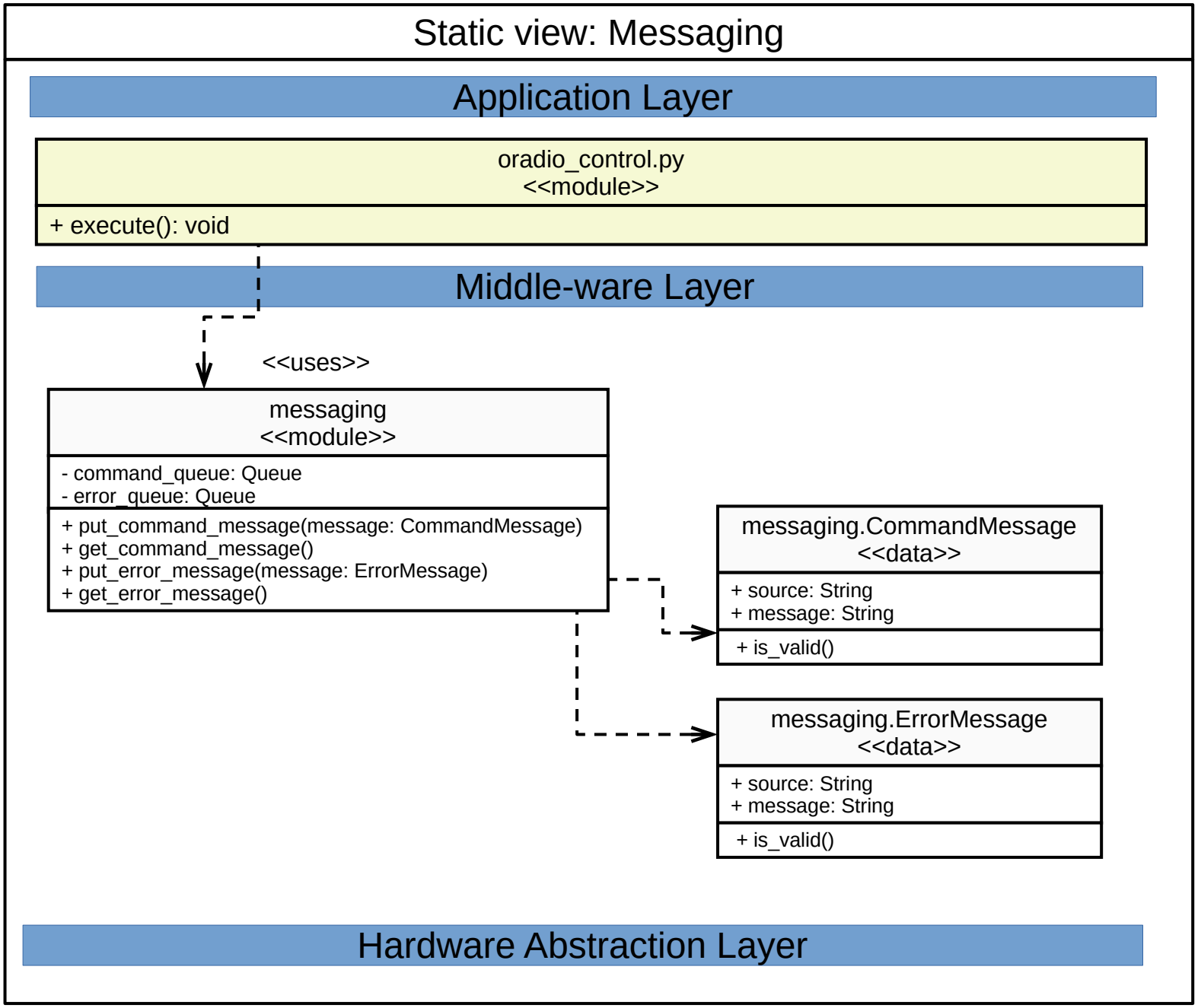

Static view: Remote monitoring
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
remote_monitoring.RMService
<<class>>
- rms_queue: Queue
+ send_message()
+ close()
<<uses>>
wifi_service.WifiService
<<Server>>
- rms_queue: Queue
+ init(queue: Queue)
+ get_state()
+ wifi_connect()
+ wifi_disconnect()
+ close()
- send_message()
<<uses>>
remote_monitoring.Heartbeat
<<class>>
+ start_heartbeat()
+ stop_heartbeat()
+ close()
<<module>>
messaging.py
+ put_error_message(message: ErrorMessage)
<<is a>>
<<BaseClass>>
Timer
- rms_queue: Queue
+ start()
+ stop()
Hardware Abstraction Layer


Static view: Web Service
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
web_service.WebService
<<class>>
- web_queue: Queue
- uvicorn_server: UvicornServer
+ init()
+ stop()
+ get_state()
+ start()
+ close()
<<consists of>>
web_server.UvicornServer
<<Server>>
+ init(app:FastAPI)
+ run()
+ start()
+ stop()
<<uses>>
fastapi_server.api_ap
<<Framework>>
+ api_app: FastAPI
+ @api_app.get()
+ @api_app.put()
+ @api_app.post()
+ @api_app.api_route()
+ @api_app.middleware()
<<uses>>
wifi_service.WifiService
<<Server>>
- web_queue: Queue
+ init(queue: Queue)
+ get_state()
+ wifi_connect()
+ wifi_disconnect()
+ close()
- send_message()
<<uses>>
<<module>>
messaging.py
+ put_command_message(message: CommandMessage)
+ put_error_message(message: ErrorMessage)
Hardware Abstraction Layer


Static view: USB
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
<<consists of>>
usb_service.USBService
<<class>>
- usb_observer: USBObserver
- usb_monitor: USBMonitor
+ init()
+ get_state()
+ close()
- usb_inserted()
- usb_removed()
- send_message()
<<consists of>>
<<singleton>>
usb_service.USBObserver
- obserrver: Observer
+ start()
+ stop()
<<uses>>
<<singleton>>
usb_service.USBMonitor
- on_created()
- usb_deleted()
<<uses>>
<<Library>>
watchdog.observers.Observer
+ schedule()
<<module>>
messaging.py
+ put_command_message(message: CommandMessage)
+ put_error_message(message: ErrorMessage)
<<is a>>
<<BaseClass>>
PatternMatchingEventHandler
+ handle_event(event)
Hardware Abstraction Layer

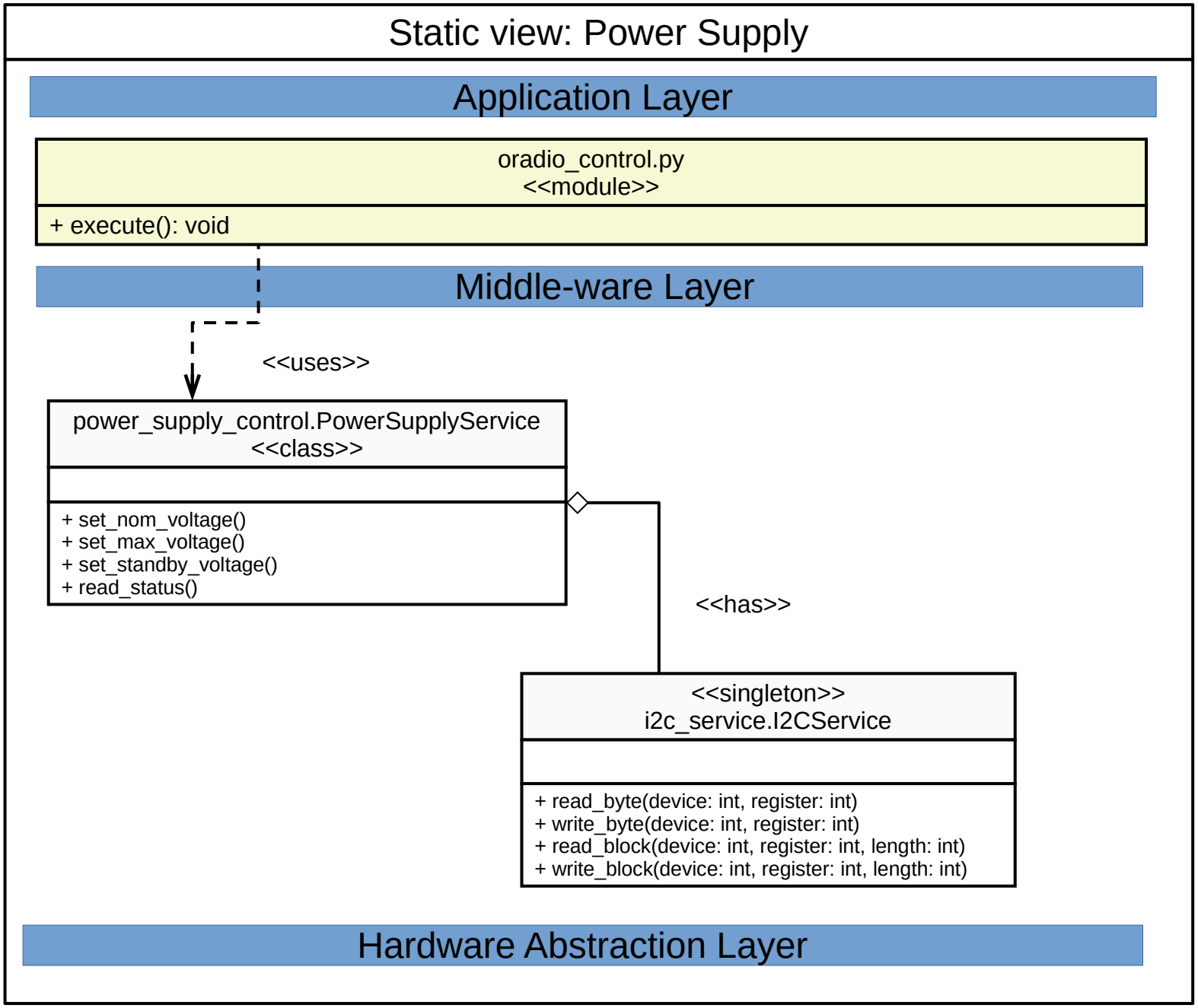

Static view: Volume
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
Waarom start(), stop(), set_notify().
Staat altijd op notify. Dus notify default
aan. Gaat nooit gestopt worden
<<uses>>
volume_control.VolumeControl
<<class>>
+ set_notify()
+ start()
+ stop()
<<has>>
i2c_service.I2CService
<<class>>
+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)
<<uses>>
<<module>>
messaging.py
+ put_command_message(message: CommandMessage)
+ put_error_message(message: ErrorMessage)
Hardware Abstraction Layer


Static view: Error
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
error_service.py.ErrorService
<<class>>
- init()
- error_handler()
from messaging import (
ErrorMessage,
get_error_message,
put_error_message,
CommandMessage,
get_command_message,
put_command_message,
)
<<uses>>
<<module>>
messaging.py
+ ErrorMessage
+ get_error_message(): ErrorMessage
Hardware Abstraction Layer

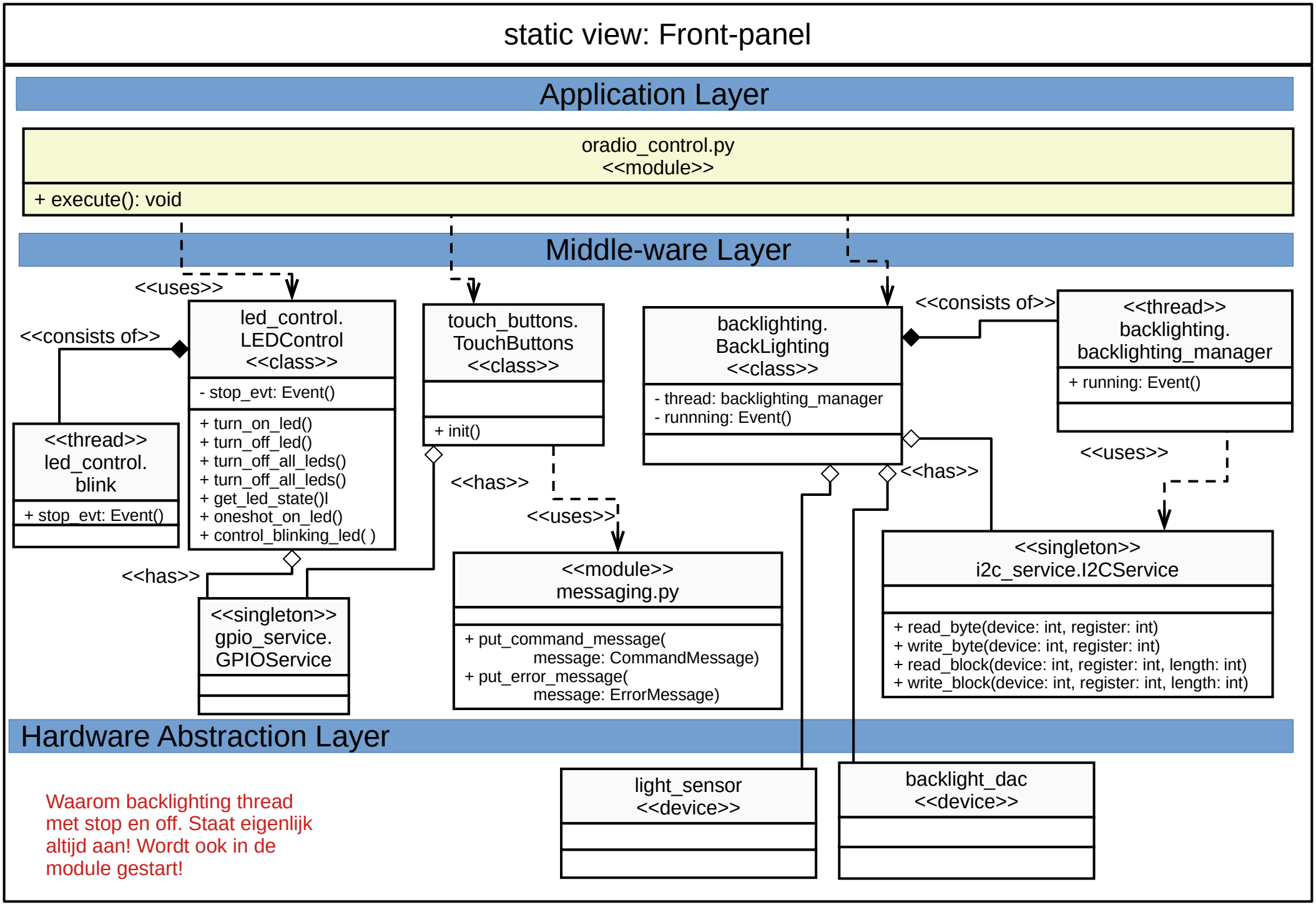


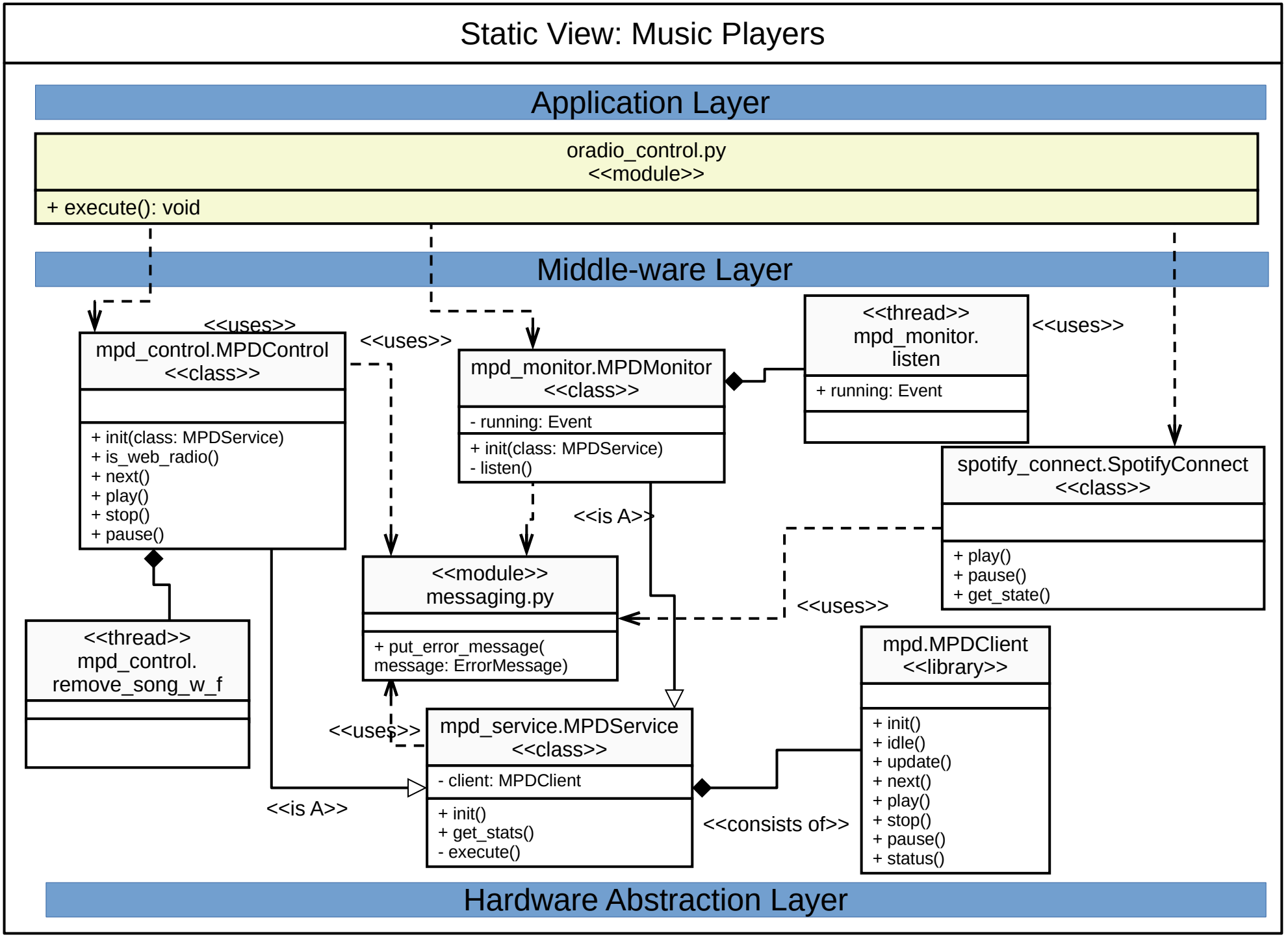
Commandmessage:
+ source: string
+ command: string
+ data: Optional[List[Any]]

Errormessage:
+ source: string
+ message: string
+ data: Optional[List[Any]]
Message: Error | Exception | ...


Static view: Power Supply
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
power_supply_control.PowerSupplyService
<<class>>
+ set_nom_voltage()
+ set_max_voltage()
+ set_standby_voltage()
+ read_status()
<<has>>
<<singleton>>
i2c_service.I2CService
+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)
Hardware Abstraction Layer

static view: Front-panel




Static View: Music Players
Application Layer
oradio_control.py
<<module>>
+ execute(): void
Middle-ware Layer
<<uses>>
<<uses>>
<<uses>>
mpd_control.MPDControl
<<class>>
+ init(class: MPDService)
+ is_web_radio()
+ next()
+ play()
+ stop()
+ pause()
mpd_monitor.MPDMonitor
<<class>>
- running: Event
+ init(class: MPDService)
- listen()
<<thread>>
mpd_monitor.
listen
+ running: Event
spotify_connect.SpotifyConnect
<<class>>
+ play()
+ pause()
+ get_state()
<<is A>>
<<module>>
messaging.py
+ put_error_message(
message: ErrorMessage)
<<uses>>
<<thread>>
mpd_control.
remove_song_w_f
<<uses>>
mpd_service.MPDService
<<class>>
- client: MPDClient
+ init()
+ get_stats()
- execute()
mpd.MPDClient
<<library>>
+ init()
+ idle()
+ update()
+ next()
+ play()
+ stop()
+ pause()
+ status()
<<is A>>
<<consists of>>
Hardware Abstraction Layer